Fake News Detection using Text Analysis and AI

Individual Project

Submitted in part fulfilment for the degree of

B.Sc. (Hons) Computing

School of Informatics and Engineering,

Technological University Dublin,

Dublin, Ireland

Dhavish Suneram

Student Number: B00152921

Supervisor: Jin Xu

May 2026

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to my supervisor Jin Xu for their continued guidance, support, and feedback throughout the duration of this project. Their advice was invaluable in shaping the direction and quality of this work.

DECLARATION

I hereby certify that this material, which I now submit for assessment on the programme of study leading to the award of Degree of Honours B.Sc. in Computing at Technological University Dublin is entirely my own work except where otherwise stated, and has not been submitted for assessment for an academic purpose at this or any other academic institution other than in partial fulfilment of the requirements of that stated above.

Signed:___Dhavish Suneram_____ Dated: 11/12/2025

ABSTRACT

This project investigates the use of text analysis and artificial intelligence to detect fake news headlines. The aim of Semester 1 was to develop a baseline system that uses TF-IDF for feature extraction and Linear Support Vector Machine (SVM) for classification. A labelled dataset of Real and Fake news headlines that was collected and pre-processed, which was then used to train the model. The resulting prototype achieves an accuracy of 75% and can classify new headlines through a simple console-based interface. In Semester 2, the system was massively extended in many ways. The dataset was expanded to the Kaggle Fake and Real News dataset, which contained approximately 44,000 labelled headlines. The LinearSVC model was also retrained on this larger dataset and achieved 95.6% overall accuracy. A DistilBERT transformer model was fine-tuned on the same dataset by using Google Colab with a T4 GPU and achieved 98.8% overall accuracy. Both of the models were evaluated by using accuracy, precision, recall, F1-score, and confusion matrix analysis. The final system was delivered as a Streamlit web application, allowing users to input any news headline and receive an instant Real or Fake prediction from either of the models.

TABLE OF CONTENTS

ACKNOWLEDGMENTS 2
DECLARATION 3
ABSTRACT 4
LIST OF TABLES 8
LIST OF FIGURES 9
ABBREVIATION LIST 10
CHAPTER 1: INTRODUCTION 11
1.1 Project Background 11
1.2 Motivation 11
1.3 Problem Statement 12
1.4 Project Aim 12
1.5 Project Objectives 12
1.6 Research Questions 14
1.7 Thesis Structure 14
CHAPTER 2: BACKGROUND RESEARCH 16
2.1 Introduction to Fake News Detection 16
2.2 Early Machine Learning Methods 17
2.3 Deep Learning and Transformer Models 19
2.4 Recent Advances with Large Language Models 21
2.5 Datasets and Evaluation Techniques 23
2.6 Current Challenges and Research Gaps 25
2.7 Summary 28
CHAPTER 3: SYSTEMS ANALYSIS 30
3.1 Methodological Approach 30
3.2 Functional requirements 32
3.3 Data Requirements 33
3.4 Technical Requirements 34
3.5 Summary 35
CHAPTER 4: SYSTEM DESIGN 36
4.1 System Architecture 36

4.2 Data Preprocessing Pipeline 37
4.3 LinearSVC Model Design 39
4.4 DistilBERT Model Design 40
4.5 User Interface Design 41
4.6 Summary 43
CHAPTER 5: IMPLEMENTATION 44
5.1 Dataset Preparation 44
5.2 Linear Support Vector Machine Model Implementation 45
5.3 DistilBERT Model Implementation 46
5.4 Streamlit Web Application 47
5.5 Summary 49
CHAPTER 6: TESTING & EVALUATION 50
6.1 Overview 50
6.2 Dataset Split 50
6.3 LinearSVC Model Results 51
6.4 DistilBERT model Results 53
6.5 Model Comparison 54
6.6 Manual Testing 55
6.7 Evaluation Against Objectives 56
6.8 Summary 56
CHAPTER 7: CONCLUSIONS & FUTURE WORK 58
7.1 Conclusions 58
7.2 Reflection 58
7.3 Limitations 59
7.4 Future Work 60
7.5 Summary 61
APPENDIX A – PROJECT PLAN SEMESTER 1 62
APPENDIX B – PROJECT DIARY SEMESTER 2 63
APPENDIX C – CODE LISTING 64
Semester 1 Code 64
Appendix C.1 – Model.py 64
Appendix C.2 Detector.py 66

Semester 2 Code 67
Appendix C.3 Prepare_dataset.py 67
Appendix C.4 Model.py (LinearSVC Model Training)..... 68
Appendix C.5 – app.py (DistilBERT prediction in Streamlit) 69
Appendix C.6 App.py (LinearSVC prediction)..... 69
REFERENCES..... 70

LIST OF TABLES

TABLE 1: CRISP-DM PHASE 31

TABLE 2: FUNCTIONAL REQUIREMENTS TABLE 32

TABLE 3: LINEARSVC EVALUATION 51

TABLE 4: DISTILBERT EVALUATION 53

TABLE 5: MODEL COMPARISON TABLE 54

TABLE 6: EVALUATION AGAINST PROJECT OBJECTIVES 56

LIST OF FIGURES

FIGURE 1: CRISP-DM CYCLE 30

FIGURE 2: SYSTEM ARCHITECTURE DIAGRAM 37

FIGURE 3: DATA PREPROCESSING PIPELINE DIAGRAM 38

FIGURE 4: LINEARSVC MODEL WORKFLOW DIAGRAM 39

FIGURE 5: DISTILBERT FINE-TUNING FLOW 41

FIGURE 6: STREAMLIT WEB APPLICATION PROTOTYPE FLOW 42

FIGURE 7: STREAMLIT WEB APPLICATION 49

FIGURE 8: STREAMLIT WEB APPLICATION 49

FIGURE 9: LINEARSVC MODEL EVALUATION METRICS IN STREAMLIT 52

FIGURE 10: DISTILBERT MODEL EVALUATION METRICS IN STREAMLIT 53

FIGURE 11: MODEL COMPARISON CHART 55

ABBREVIATION LIST

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
BoW	Bag of Words
CNN	Convolutional Neural Network
CRISP-DM	Cross-Industry Standard Process for Data Mining
CSV	Comma-Separated Values
DeBERTa	Decoding-enhanced BERT with Disentangled Attention
GPU	Graphics Processing Unit
LLM	Large Language Model
LSTM	Long Short-Term Memory
ML	Machine Learning
NLP	Natural Language Processing
RNN	Recurrent Neural Network
RoBERTa	Robustly Optimised BERT Pretraining Approach
SVM	Support Vector Machine
TF-IDF	Term Frequency-Inverse Document Frequency
UI	User Interface
XAI	Explainable Artificial Intelligence
XLNet	Generalised Autoregressive Pretraining for Language Understanding

CHAPTER 1: INTRODUCTION

1.1 Project Background

Fake news has become a serious problem in today's world. A lot of people are getting their news from social media, which allows false or misleading information to spread quickly before anyone has even the chance to check if it's true. Most people often share headlines without verifying them, which can have a major influence on public opinion and can cause real harm. Trying to fact-check every headline is not realistic for a person, which is why researchers have begun using artificial intelligence and natural language processing to build systems that can detect the fake news automatically.

This project examines how machine learning can be used to classify news headlines that are Real or Fake. In Semester 1, a basic prototype was developed that used TF-IDF and a Linear SVM classifier, which achieved an overall accuracy of 75% on a small dataset. In Semester 2 this was improved substantially by using the Kaggle Fake and Real News dataset, which had 44,000 headlines, which pushed the LinearSVC model to have an accuracy of 95.6%. A DistilBERT transformer model was also fine-tuned on the dataset and achieved an overall accuracy of 98.8%. The final system was built by using a Streamlit web app where users can type in headlines and get an instant prediction.

1.2 Motivation

It is getting much harder for everyday people to tell what Real news is and what is Fake. Headlines are often short and emotional, which makes them easy to share and can be easy to manipulate. A simple tool that can instantly tell a user whether a headline is likely Real or Fake could help people make better decisions about what they read and share online.

This project also gives the opportunity to apply knowledge gained throughout the course to a real-world problem, including machine learning, text processing, and web development. It also allowed to compare a simple baseline model against a more advanced transformer model to see how much of a difference the new technology makes.

1.3 Problem Statement

People tend to often judge news based on headlines alone, even though headlines can be misleading or even completely made up. Most detection tools that exist today are too complex for everyday users. This project tries to answer the following question:

How can an AI system analyse a news headline and decide whether it is Real or Fake, and how does a basic machine learning model compare to a modern transformer model for this task?

1.4 Project Aim

The aim of this project is to build, test, and evaluate a fake news detection system that uses text analysis and AI to classify the headlines to see if they are Real or Fake and to deliver this as an easy-to-use web application for users.

1.5 Project Objectives

To achieve this aim, a set of clear measurable objectives was defined at the start of the project. These objectives formed every decision throughout, from the data that was collected and processed, to which models were chosen and how the final-system was built and further tested.

1. Collect and prepare a large, labelled dataset of a Real and Fake news headlines, to ensure it is balanced and is large enough to train reliable models.

2. Build and evaluate a LinearSVC model using TF-IDF with unigram and bigram features, and assess the performance of the full dataset

3. Fine-tune a DistilBERT transformer model on the same dataset and compare its performance directly against the LinearSVC baseline

4. Evaluate both models using accuracy, precision, recall, F1-score, and confusion matrix analysis, to give a complete picture of the performance

5. Build a Streamlit web application that allows any user to enter a news headline and receive a real-time Real or Fake prediction from both models

6. Critically evaluate the strengths and weaknesses of the final systems and identify areas where further development would improve its real-world usefulness.

Each of these objectives is addressed across the following chapters that follow: The dataset collection and preprocessing are covered in Chapters 3 and 5, the model design and implementation in Chapters 4 and 5, the evaluation results in Chapter 6, and the critical reflection in Chapter 7. Together, they form a structured path from problem to solution.

1.6 Research Questions

The objectives above led to four research questions that guided the direction of the project. These research questions helped me to focus on the work and gave a clear idea of what a successful outcome would look like:

1. Can a TF-IDF and LinearSVC model reliably classify fake news headlines when trained on a large dataset?

2. How much does fine-tuning DistilBERT improve accuracy when compared to the LinearSVC baseline, and is the extra complexity justified?

3. What are the main limitations of using text alone to detect fake news, and where does each model struggle the most?

4. How can the final system be made in an accessible way that makes it useful for everyday users with no technical background?

These questions are revisited and answered in the evaluation chapter, where the results for both models are discussed in detail. They also inform the reflection and future work sections in Chapter 7.

1.7 Thesis Structure

This thesis is structured across seven different chapters, with each building on the previous one to take the reader from the problem background through to the conclusion.

Chapter 1 introduces the project, which explains why fake news detection matters, what the system aims to do, and the research questions that also guided the work. Chapter 2

reviews the existing academic literature on fake news detection, covering early machine learning approaches, deep learning models, transformer models such as BERT and the most recent work using large language models. This background research directly informed the decision to use both LinearSVC model and the DistilBERT model in this project, as it allowed for a meaningful comparison between a classical and a modern approach.

Chapter 3 covers the system analysis, which explains the methodological approach used (CRISP-DM) and setting out the functional, data, and technical requirements the system needed to meet. Chapter 4 moves from the requirements into the design, which describes how the system was structured, how data flows through the preprocessing pipeline, and how both models were designed before implementation even began.

Chapter 5 describes the full implementation of the system, covering dataset preparation of both of the models' implementations, and the Streamlit web application. Chapter 6 presents the testing and evaluation results for both models when comparing them across a range of metrics and discussing the strengths and weaknesses of each. Finally, Chapter 7 draws conclusions from the project which reflects on the overall experience and outlines the area for future development.

CHAPTER 2: BACKGROUND RESEARCH

2.1 Introduction to Fake News Detection

It is easier than ever to spread information because of the development of online news and social media, and it has also created more opportunities for erroneous or misleading information to spread far and wide. False news is capable of altering the public opinion, influencing emotions, and affecting real-life events, which is why the issue of its detection has become a significant area of research in the context of artificial intelligence (AI) and natural language processing (NLP) (Oshikawa, Qian and Wang, 2020; Shu et al., 2017). This review examines prior approaches to fake news detection to identify their strengths, limitations, and the remaining gaps that motivate the development of more reliable AI-based detection methods.

One of the main reasons behind the identification of fake news is the fact that deceptive articles tend to be modelled in the tone, structure, and writing style of authentic journalism. Because they are formulated to sound persuasive, it might be hard to tell whether the information is authentic to both human and automated systems (Zhou and Zafarani, 2020). Scholars explore this problem from different angles. Shu et al. (2020) concentrates on the transmission of misinformation by users and social behaviour on the Internet, and Zhou and Zafarani (2020) concentrates on linguistic signs and patterns that can be used to suspect potentially deceptive writing. These two perspectives demonstrate that fake news detection requires considering both the content of a text and the social context in which it is provided. Together, these studies show that effective fake news detection cannot rely on a single type of evidence. Linguistic cues alone are often insufficient, while social-context signals can be unreliable on their own; Combining both perspectives leads to more comprehensive detection strategies.

Because these challenges could not be addressed with simple rule-based systems, researchers initially turned to classical machine-learning methods, which are discussed in the next section.

2.2 Early Machine Learning Methods

Before deep learning became widely used, researchers mainly relied on classical machine learning (ML) techniques to detect fake news. These early systems treated the task as a straightforward text-classification problem, where the model learned from features that were manually created by researchers. Algorithms such as Support Vector Machines (SVM), Naïve Bayes, Decision Trees, and Logistic Regression were common choices because they were quick to train, easy to work with, and simple to interpret (Ahmed, Traore and Saad, 2018).

To turn text into something a model could analyse, researchers usually use Bag-of-Words (BoW) or Term Frequency - Inverse Document Frequency (TF-IDF). These methods convert words into numerical values based on how often they appear in the text. While these approaches are straightforward and work well on highly controlled datasets, they do not capture the meaning of words or their relationships (Wang, 2017).

Performance across studies generally ranged from 65% to 80% accuracy, though results depended heavily on the dataset used. For example, SVMs performed well on shorter political utterances in the LIAR dataset, whereas Naive Bayes was more successful in more balanced datasets when the ratio of Fake to Real news was equal (Wang, 2017). On the contrary, FakeNewsNet, which provides articles along with data on social-media interactions, exhibited more unpredictable behaviour due to the variety of its content, which is more difficult to model (Shu et al., 2017). These discrepancies indicate a more

serious issue: classical ML models tend to underperform when generalising to new subjects, writing styles, or platforms.

Although these models provided a useful starting point, they also come with clear limitations:

• They struggled with vocabulary changes, so two sentences with similar meanings but different wording could confuse the model.

• They were highly topic-dependent, performing well on one subject but poorly on another.

• Even small paraphrasing could reduce accuracy because BoW and TF-IDF don't capture deeper meaning

• Much of the improvement depended on manual feature engineering, meaning researchers had to design input features themselves rather than having the model learn them automatically.

• Cross-domain performance was weak, with models often doing well on one dataset but poorly on another.

Despite these constraints, classical ML approaches are still used as a control group today because they are fast, cheap, and interpretable (Raza et al., 2024; 2025). They are still useful for comparing older techniques with advanced ones to demonstrate the extent to which newer models have evolved.

But the problems of these primitive ways led to the conviction that something greater was wanted. Such difficulties motivated researchers to shift to deep-learning models, which can learn meaning, context, and more elaborate language patterns. The following section examines this change to deep-learning and transformer-based methods.

2.3 Deep Learning and Transformer Models

As fake news grows more sophisticated and difficult to identify, researchers have started to resort to deep-learning-based models that have managed to identify the patterns that traditional machine learning algorithms could not. Initial research in this field had centred on neural network architectures such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs). The use of Long Short-Term Memory (LSTM) networks, particularly, has become popular as it has the capability to process text sequentially and to remember significant information about previous words, and allows the model to learn writing patterns or linguistic clues that are associated with deceptive content (Ajao, Bhowmik, and Zargari, 2019). The results of the studies demonstrated that CNN and LSTM models were much more effective than SVM and Naive Bayes classifiers and performed significantly better on longer news articles (Ahmed, Traore and Saad, 2018).

Performance improved even further with the development of word-embedding models such as Word2Vec and GloVe. Rather than simply counting words, these embeddings represent the semantic relationship between words by placing similar meanings closer together in vector space. This allows models to capture context and meaning far more effectively than traditional Bag-of-Words or TF-IDF approaches (Zhou and Zafarani, 2020). When used with CNN or LSTM architectures, word embedding consistently

produced more accurate and stable results across different datasets (Kaliyar, Goswami and Narang, 2021).

One breakthrough was the introduction of transformer-based models, particularly BERT (Bidirectional Encoder Representations from Transformers). Unlike older models that read text in only one direction, BERT reads in both directions simultaneously, and that enables it to receive the full scope of the context and the profound meaning of the sentences. Researchers achieved state-of-the-art results by fine-tuning pre-trained BERT models on datasets like LIAR, ISOT, and FakeNewsNet (Oshikawa, Qian, and Wang, 2020). Transformers also did not rely, as RNNs did, on the sequential processing bottleneck, though they consume more memory and computational power and perform well at learning long-range patterns and dependencies. Later transformer models further improved:

• RoBERTa refined BERT by removing the next-sentence prediction step and using more robust pre-training.

• XLNet introduced permutation-based training to capture richer contextual relationships.

• DeBERTa enhanced attention mechanisms to better differentiate important from unimportant information.

Across recent studies, these models consistently outperform CNNs, LSTMs, and earlier versions of BERT, especially when dealing with complex or noisy text (Raza et al., 2024; Hu et al.,2024).

Nevertheless, the transformer models are not without restrictions. They are also costly to train on long documents, as their attention mechanism is quadratic. They are also not transparent; their inner logic is hard to decode, so they are often treated like black boxes. Due to this, more and more researchers are investigating explainable AI (XAI) techniques to determine which words or phrases affect model decision (Alshuwaier and Alsulaiman, 2025)

Altogether, the combination of deep learning and transformer models has pushed fake news detection well beyond word-frequency models, enabling the model to decode context and semantics, as well as more profound relationships within the text. Their good results are the basis of the latest generation of detection techniques, such as large language models. These new developments are discussed in the following section.

2.4 Recent Advances with Large Language Models

Building on the progress made by transformer architecture, recent research has increasingly focused on Large Language Models (LLMs), which extend these capabilities even further.

Recently, Large Language Models (LLMs) like GPT-3.5, GPT-4, LLaMA and RoBERTa-large have become significant instruments in fake news detection. In contrast to previous deep learning models, LLMs are trained on very large, diverse text corpora, enabling them to comprehend language at a much deeper semantic and contextual level. A significant strength of these models is that they can do zero-shot and few-shot classification. That is, they are able to detect false information with just a bare prompt or only a few examples, without any training devoted to the particular subject of misinformation or the rapid emergence of false information.

It is also becoming frequent to fine-tune smaller LLMs. Smaller models like LLaMA-2-7B and 13B, when fine-tuned on datasets like LIAR, ISOT or COVID-19 misinformation corpora, are frequently very strong performers, and they consume significantly fewer computational resources than larger models (Hu, 2025). In other words, combining the outputs of LLMs with classic machine-learning characteristics or social-media credibility cues can generate more reliable predictions (Gong et al., 2024). Altogether, the findings of the recent studies demonstrate that LLMs perform better than the previous models, especially in activities that require long-form writing, emotional framing, and minor discrepancies in writing.

Despite these advantages, LLMs have several critical limitations. Among others is the problem of prompt sensitivity, where minor variations in how a question is worded can produce varied or unstable results. This may complicate the ability to ensure consistency of predictions across cases. The other issue is bias because the LLMs train on the training data and can accidentally reproduce and propagate dangerous or misleading opinions. Hallucinations may also be produced by LLMs, in which the model comes up with explanations or classifications that are logically plausible but are factually false. It is especially dangerous in misinformation detection, where a hallucinated reason will strengthen false statements instead of putting them into question.

There are also security issues that have been raised. LLMs are also vulnerable to adversarial text attacks, in which minor, well-thought-out word manipulations can lead the model to think that Fake news is Real or vice versa. Such shortcomings have raised concerns about explainable AI (XAI) methods. The recent literature examines the approaches aimed at highlighting the words or phrases that led to a model decision and

assisting the users to comprehend how the model reached its output and to trust the system more (Alshuwaier & Alsulaiman, 2025).

Although most studies report strong performance from LLMs, others highlight that these gains often depend heavily on prompt design and dataset quality, suggesting their advantages are not universal.

On the whole, LLMs have become important tools for fake news detection, as they provide more convincing reasoning, better contextual awareness, and greater adaptability than older models. But their shortcomings, such as bias, prompt instability, hallucinations, and susceptibility to manipulation, indicate that they still require careful consideration and prudent use. The next section reviews the dataset and evaluation techniques used to measure the performance of these models.

2.5 Datasets and Evaluation Techniques

The datasets are in the centre of the research of fake news detection, as the quality, the variety of data and its structure directly influence the performance of a model. The various datasets are concerned with different forms of misinformation, and this implies that the performance of a model can be vastly different when it is trained on different things. The initial datasets include short political statements tagged on a six-point truthfulness scale, like the LIAR dataset (Wang, 2017), and can be used to assess classical machine learning models. FakeNewsNet (Shu et al., 2017) adds to this, where full news items are included in addition to social media interaction, allowing models to analyse the linguistic aspects of information as well as user-interaction behaviour. The ISOT dataset (Ahmed and Saad, 2018) provides longer, properly structured articles divided into Real and Fake, which is convenient in terms of deep learning and transformer models.

Newer datasets deal with contemporary issues of misinformation. CoAID and ReCOVery, as an example, address COVID-19-related information and contain article text, social reactions, and metadata (Cui and Lee, 2020). Also, common these days are multilingual datasets, which are a sign of the global spread of misinformation and enable researchers to test whether models generalize across languages (Raza et al., 2024). The newer datasets are larger and more varied and are better suited to be trained on LLMs and advanced transformer models.

Although they are useful, the current datasets have a number of weaknesses. Most of them are politically biased and concentrate more on U.S. content, making it difficult to generalize it to other issues like health, finance, or international news. Others are outdated, particularly those that were developed prior to the COVID-19 pandemic or before the widespread use of LLMs. Label quality is also an issue: As many datasets rely on crowd-sourced or loosely verified labels, this reduces the reliability of model evaluation. Well-edited collections may still tend to reduce misinformation to binary (Real vs Fake) and may fail to identify satire or partially true statements and context-dependent claims.

Due to the large variability in structure and difficulty of fake news datasets, it is typical to find models that perform well on one dataset but poorly on another. This complicates the process of evaluation, and it underscores the importance of testing models with a wide range of datasets in order to have a realistic view of their advantages and limitations.

Measurements of evaluation are also important. Even though accuracy is the most frequently reported score, it is deceiving, particularly when there is an imbalance in datasets, which is typical in misinformation studies (Panwar et al., 2024; Kumari and

Singh, 2025). Precision, recall, F1-score, macro/micro averaging, and AUC provide a more comprehensive view of how the models behave. Macro-averaged F1 is especially important for fake news datasets because it prevents models from appearing strong when they mostly predict the majority class correctly. To demonstrate this, such high accuracy could conceal that a model infrequently predicts fake news correctly when the data has almost all real articles. The fact that various measures are used contributes to the exposure of these shortcomings and will result in a more just assessment.

In general, to create a solid fake news solving framework, it is crucial to pay close attention to the selection of datasets and have a solid evaluation plan. These datasets and evaluation limitations connect directly to the broader challenges faced by fake news detection systems, which are discussed in the next section.

This project uses the Kaggle Fake and Real News dataset, which contains 44,000 labelled news headlines. The selected dataset is well balanced between Real and Fake examples and is also widely used in the research community, which makes it a very reliable choice for training and evaluating both of the models.

2.6 Current Challenges and Research Gaps

The challenges in fake news detection can be grouped into four main categories: data-related issues, model-level limitations, evaluation concerns, and real-world deployment constraints.

Although the level of fake news detection has greatly improved with deep learning, transformer models, and large language models, the systems encounter a number of issues that make them not fully reliable in real-life applications. A dataset's quality is one of the

most frequent problems. Most popular datasets are small, out-of-date, or only address U.S. political content, which restricts the ability of models to generalize to other areas like world events, finance, or health misinformation (Zhou and Zafarani, 2020). In other datasets, there also exists label bias, in which the concept of being Fake or Real is not consistent, and models find it more difficult to learn to make the correct predictions (Shu et al., 2017).

The second significant difficulty is the lack of generalisation between domains. A model can be very good with one set of data and fail with a different writing style, language, or subject. This is particularly apparent when the misinformation is composed with the purpose of deceiving or concealing the false information in sarcasm or inappropriate wording. The contemporary version of LLM continues to struggle with adjusting to these changes (Raza et al., 2024; Hu, 2025).

Explainability is another critical problem. Most AI models, in particular transformers and LLMs, are designed to act as black boxes, providing predictions without explicitly demonstrating the process by which they arrived at the result. This renders automated detection unreliable to journalists, policymakers, and ordinary users. Whereas the emergence of new explainable AI methods emphasizes the presence of influential words or sentences, they remain insufficient and do not always give a complete picture of how the LLMs make their choices (Gong et al., 2024).

The problem of multilingual and cross-cultural misinformation also poses numerous challenges to fake news. The bulk of datasets and research is dedicated to the English language, despite the fact that misinformation is widespread in a variety of languages and cultures. The use of models that are trained using English content has generally failed

when it is applied to other regions because of varied writing styles as well as linguistic structures. The lack of quality bilingual and multilingual data sets is one of its largest gaps (Alsulaiman, 2025).

A more recent problem is security and adversarial attacks. Even minor, well-crafted alterations of text, like swapping some words or changing sentence structure, can make both transformers and LLMs get Real and Fake news mixed up and are specifically created to go undetected (Hu, 2025).

Lastly, there is the problem of detecting in real time. Most systems are linked to entire news stories, though fake news on such platforms as TikTok, X/Twitter, or Instagram is disseminated in brief posts, memes, pictures, and mixed media. Speedy identification of fake material prior to its dissemination is a challenging and mostly unsolvable dilemma (Cui and Lee, 2020).

Generally, the field is yet to fill gaps in dataset quality, generalisation, multilingual support, explainability, and resilience to adversarial manipulation. These loopholes indicate that detection systems cannot be entirely relied upon in real-life contexts until more studies are conducted. The given project will help fill in some of these challenges by examining a more robust and generalizable text-based detection method based on recent AI methods. This motivates the design choices in this project, which focuses on improving generalisation and practical reliability using modern text-based AI models.

2.7 Summary

Fake news detection has evolved rapidly over the past decade, shifting from straightforward machine-learning models with hand-crafted features to more sophisticated deep-learning and transformer-based models. First-generation methods like SVM, Naive Bayes, and logistic regression were quite decent and lacked the ability to capture context, semantics, and relationships between words. Deep learning models, such as CNNs and LSTMs, reduced these shortcomings by better learning richer language patterns, and transformer models, such as BERT, RoBERTa, XLNet, and DeBERTa, did even better by learning long-range dependencies and bidirectional context. In recent years, Large Language Models (LLMs) have gone a step further to expand the field by providing zero-shot detection functionality but have also presented new issues like prompt sensitivity, hallucinations, and bias.

The quality of data and the methods of evaluation have contributed significantly to these developments. Even though newer and more diverse datasets have been able to improve model performance, problems of dataset bias, label noise, multilingual coverage, and inconsistent evaluation measures still exist. The discipline still encounters serious obstacles, such as poor cross-domain generalisation, poor explainability, susceptibility to adversarial manipulation, and inability to detect fast-moving social media on a real-time basis.

In general, fake news detection is an active and developing field of research. Even though the current AI techniques have become highly accurate and resilient, additional efforts are required until these systems gain complete trust in real-life situations. This review forms the basis of the subsequent chapter outlining the methodology applied in this project, which consists of dataset collection, text processing procedures, and the modelling approach to identify fake news with the help of text analysis and AI.

This research that was done in this chapter helped to decide to use two models in this project. LinearSVC was selected as a simple and fast baseline and DistilBERT as a more advanced transformer model. This allowed to compare a classical machine learning approach with a modern transformer approach to see which one works better for fake news detection.

CHAPTER 3: SYSTEMS ANALYSIS

This chapter outlines the methodological approach taken in this project and defines the requirements that the system needed to meet. Rather than jumping straight into building, it was very important to first understand the problem clearly, decide how to approach it, and agree on what the system actually needed to do before any design or implementation work began.

3.1 Methodological Approach

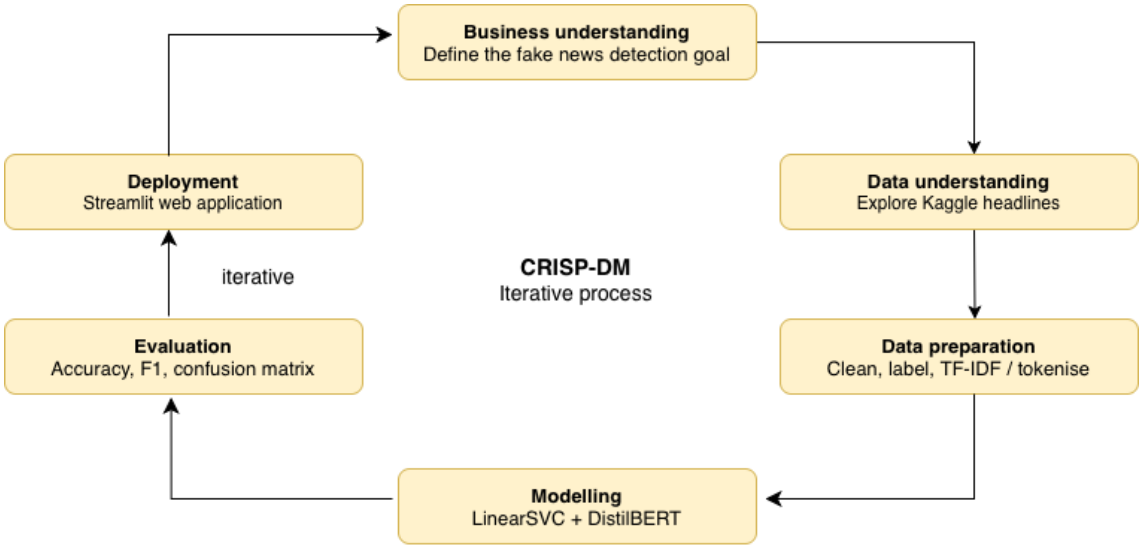

Figure 1: CRISP-DM Cycle

This project follows the CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology. CRISP-DM is a widely used framework for machine learning and data mining projects because it provides a clear structured process that covers everything from understanding the problem right through to deployment. It was chosen here because it fits naturally with the way a fake news detection system needs to be developed. The data, the model and deployment all need to be thought about together rather than in isolation.

Table 1: CRISP-DM Phase

CRISP-DM Phase	**How it applies to this project**
Business Understanding	The goal is to detect Fake vs Real news headlines quickly so users can verify information that they read. The system should be simple to use and give fast predictions.
Data Understanding	A labelled dataset of Real and Fake headlines was explored. Key challenges identified included the short length of headlines, limited context, and how similar Real and Fake news headlines can look
Data Preparation	Texts that were cleaned by lowercasing, removing symbols, and extra whitespace, and then converted into numerical features by using TF-IDF vectorisation
Modelling	A LinearSVC classifier was trained using the TF-IDF features. In Semester 2 the DistilBERT transformer model was also fine-tuned on the same dataset for comparison
Evaluation	Both of the models were tested on a held-out set which used accuracy, precision, recall, F1-score, and confusion matrix analysis
Deployment	In Semester 1, a console-based type prototype was built. In Semester 2 this was replaced with a fully functional Streamlit web application.

One of the key advantages of the CRISP-DM is that it is iterative, so if something does not work at the modelling or evaluation stage, it is expected to go back and revisit the data preparation or even the problem definition. This was very useful in Semester 2 when

the dataset was expanded and the second model was introduced, as the process allowed for those changes to happen without disrupting the overall structure of the project.

3.2 Functional requirements

The functional requirements define what the system must be able to do from the users' perspective. These were identified early in the project to make sure the final system would actually be useful and not just technically accurate.

Table 2: Functional Requirements Table

Requirement	Description	Priority
REQ-01	Accept a news headline as text input from the user	High
REQ-02	Preprocess input using lowercasing, symbol removal and TF-IDF vectorisation	High
REQ-03	Classify the headline as REAL OR FAKE	High
REQ-04	Support two models: LinearSVC and DistilBERT with user selection	High
REQ-05	Displays the prediction results clearly to the user	High
REQ-06	Display evaluation metrics: accuracy, precision, recall, F1- score, and confusion matrix	Medium

These requirements stayed consistent across both semesters, though the implementation changed significantly. In Semester 1, requirements REQ-01 to REQ-3 were met through a basic console interface. By the end of Semester 2, all six of the requirements were fully met through the Streamlit web application.

3.3 Data Requirements

For the next classification system like this one, the quality and size of the training data have direct impact on how well the models perform. The data requirements for these projects were:

- The dataset must be labelled, with each headline clearly marked as either real or fake.

- The dataset must be large enough to train reliable models; small datasets were identified in the literature review as a key weakness of early fake news detection systems.

- The dataset must be reasonably balanced between Real and Fake examples to avoid the model simply learning to predict the majority class.

- The data must consist of headlines rather than full articles to reflect how users encounter news online and to reduce complexity.

The Kaggle Fake News and Real News dataset was selected because it meets all of these requirements. It contains approximately 44,000 labelled headlines that are split evenly between Real and Fake news, and it is widely used in the research community, making it a reliable and well-understood choice. The dataset was split into 67% training and 33%~ testing, using a fixed random state to ensure that the results could be reproduced.

3.4 Technical Requirements

The following tool and technologies were required to build and run the system:

- Python 3 – main program language used throughout the project
- scikit-learn – used to implement the TF-IDF vectoriser and the LinearSVC classifier HuggingFace Transformers – used to load and fine-tune the DistilBERT model
- PyTorch – Required for running the DistilBERT model interface
- Streamlit - used to build the web application interface
- joblib – used to save and load the trained model LinearSVC model so it does not need to be trained on every run
- Google Colab with T4 GPU - required for fine-tuning DistilBERT, as the model is too large to train efficiently on a standard laptop.

These tools were chosen because they are all widely used in the machine learning community, well-documented, and free of use. The combination of scikit-learn for the baseline model and HuggingFace for the transformer model also directly reflects the comparison made in the literature reviews between classical and modern approaches.

3.5 Summary

This chapter sets out the methodological and requirements foundation for the project. The CRISP-DM methodology provided a structured approach that was flexible enough to accommodate the changes made in Semester 2. The functional, data, and technical requirements defined here guided in every decision made in the design and implementation phase that follow. Chapter 4 now moves on to describe how the system was actually designed based on these requirements.

CHAPTER 4: SYSTEM DESIGN

This chapter will describe how a fake news detection system was designed. With the requirements set out in Chapter 3, the focus here is on the decisions made about how to structure the system, how the data moves through it, and how each of the two models was designed before implementation began. The design covers four main areas, which are the overall system architecture, the data preprocessing pipeline, the design of each model, and the user interface.

4.1 System Architecture

The overall system consists of four main components that work together in sequence. A user submits a news headline through the interface. The headline is then pre-processed to convert int into a format the model can work with. The selected model can work with. The selected model makes a prediction, and the results is then displayed back to the user.

In Semester 2, the system architecture was extended to support two separate models, the LinearSVC model and the DistilBERT model, both integrated into a single Streamlit web application. The user can switch between both models, and the application handles all preprocessing, prediction, and display logic internally. Both models are loaded using caching, so they only need to be initialised once per session, which keeps the response times fast.

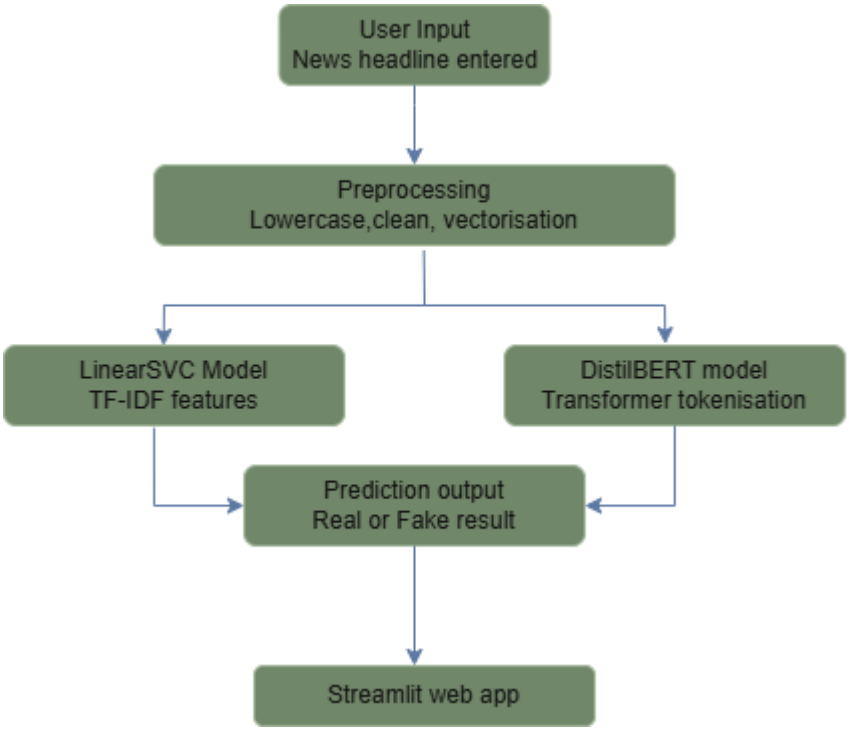

Figure 2: System Architecture Diagram

This diagram shows the complete workflow used during model training and user prediction.

4.2 Data Preprocessing Pipeline

Before any headline can be processed by a model, it needs to be cleaned and converted into numerical format, as raw text cannot be used directly by machine learning models, so the preprocessing pipeline was designed to handle this transformation consistently for both training and prediction.

For my LinearSVC model, the preprocessing pipeline work as follows: input text is first converted to a lowercase to remove case inconsistencies, then any symbols and extra whitespaces are then removed, and finally TF-IDF (Term Frequency- Inverse Document frequency-based vectorization was then applied to convert the cleaned text into numerical feature vectors. The TF-IDF vectoriser was configured to ignore common English stop

words such as “the”, “and” to learn both single words and two-word phrases (unigrams and bigrams) and to filter out every rare or very common term that would add noise without helping classification.

For the DistilBERT model, the preprocessing is handled differently. Rather than TF-IDF, the headlines are tokenised by using the DistilBERT tokenizer, which converts the text into token IDs that the transformer model can interpret. A maximum sequence length of 128 tokens was set, which is sufficient for the headlines, and it also avoids unnecessary computational overhead.

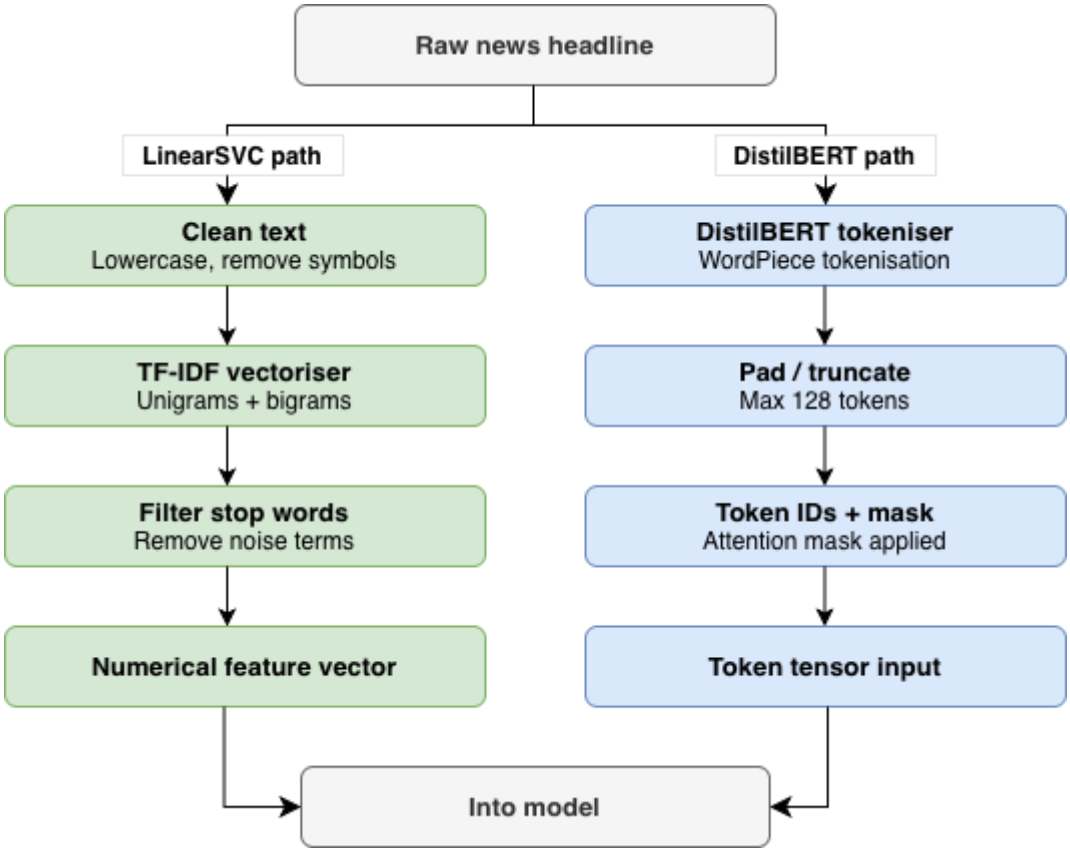

Figure 3: Data Preprocessing Pipeline Diagram

The result is numerical representation of the headline that can be processed by the SVM classifier.

4.3 LinearSVC Model Design

The LinearSVC model was selected as the baseline because it is well suited to short text classification tasks like this one. It handles the high-dimensional feature space, which is produced by TF-IDF, effectively; trains quickly without requiring a GPU and has been widely used as a strong baseline in the news detection literature.

The model was designed as a single pipeline combining the TF-IDF vectoriser and the LinearSVC classifier into one object. This means preprocessing and classification will happen in one step, which makes the code cleaner and avoids any risk of applying different preprocessing at training time versus prediction time. The pipeline was trained on 67% of the dataset, and the trained model was saved to disk so it could be loaded by the web application without any retraining.

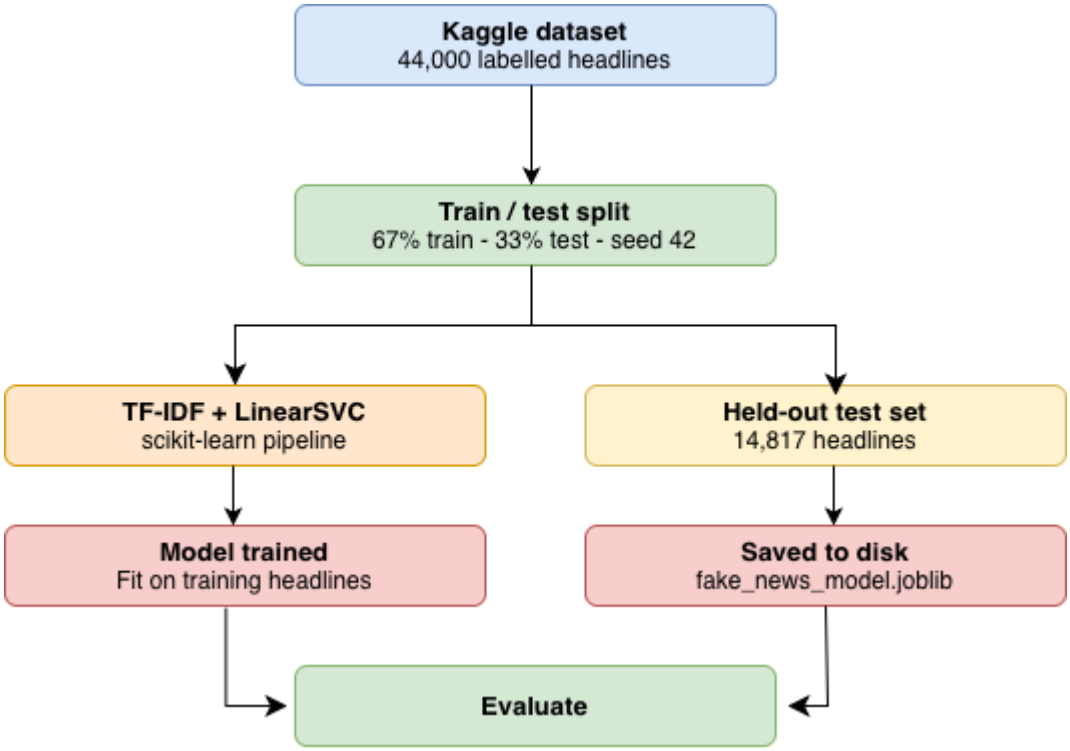

Figure 4: LinearSVC Model Workflow Diagram

This flow shows how the numerical features generated from text are passed through the SVM model to produce a final classification.

4.4 DistilBERT Model Design

DistilBERT was selected in my project as the advanced model because it provides a strong balance between performance and practicality. It has a lighter version of the original BERT model, retaining around 97% of BERT's language understanding capability while being 40% smaller and 60% faster. This made it a lot more suitable for deployment in a real-time application than full BERT.

The model was designed around the distilbert-base uncased version from HuggingFace. Which was fine-tuned on the same Kaggle dataset used to train the LinearSVC model. Fine-tuning means that the model weights, which were already pre-trained on a large general text corpus, are then updated further using the fake news dataset. This allows the model to apply its general understanding of language to the specific tasks of fake news classification.

The fine tuning was done on Google Colab by using a T4 GPU runtime due to the model being too large to train efficiently on a standard machine. After the training was completed, the model files were saved and downloaded for my web application. The design is to keep both models as an option in my application, rather than choosing just one, so that the users can directly compare predictions, and the interface can also display evaluation metrics for both sides of the models.

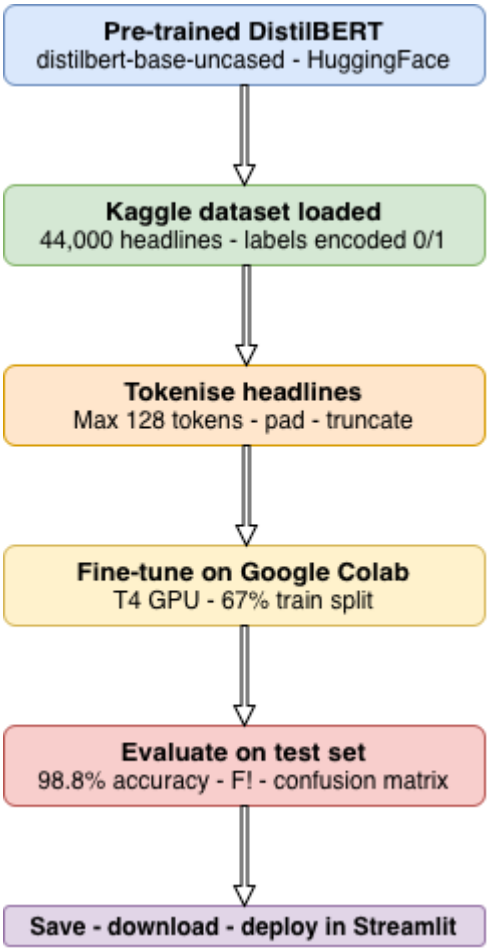

Figure 5: DistilBERT Fine-tuning Flow

4.5 User Interface Design

The user interface was designed with pure simplicity, as the target users are not supposed to have a major technical background, so the interface should be very clear, fast and efficient to use without any instructions.

In Semester 1, the interface was just a basic console -based prototype where the users typed a headline and received an immediate Real or Fake prediction. This was sufficient in terms of testing the model, but not suitable as a final product.

In Semester 2, the interface was replaced with a Streamlit web application. The design includes a text input area where the users can paste any headline, a model selector to choose from between LinearSVC and DistilBERT, a color-coded prediction result, and a metrics panel that shows the accuracy, precision, recall, F1-score, and confusion matrix for both models in separate tabs. A dark theme was also used to improve users' readability and to give the application a cleaner user experience.

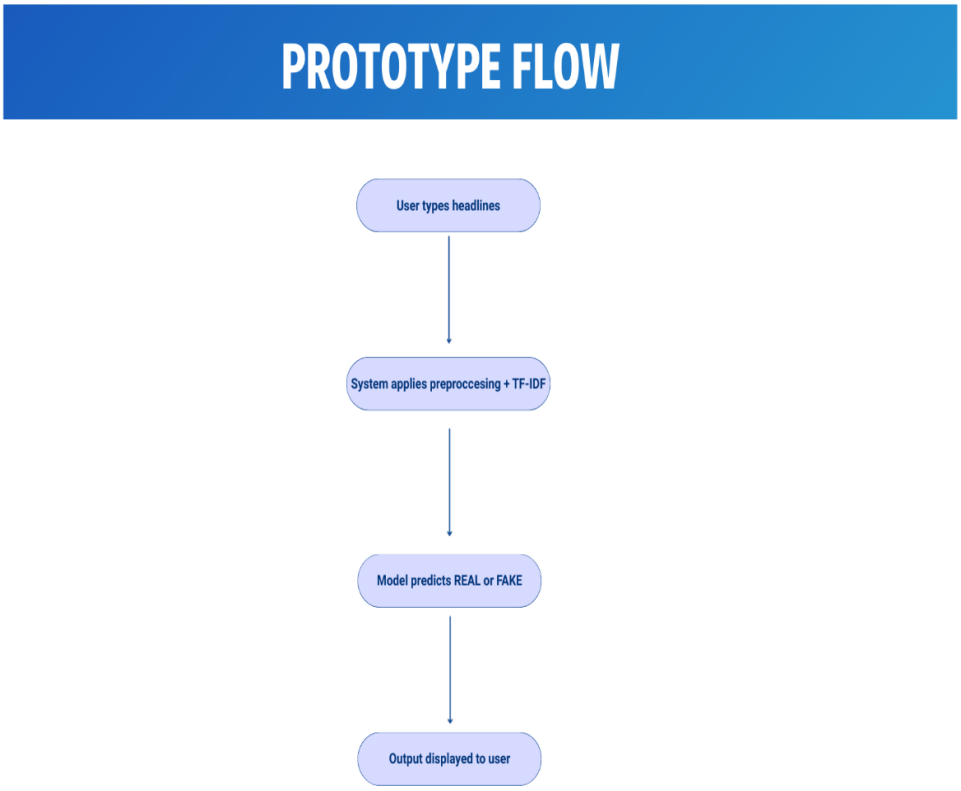

Figure 6: Streamlit Web Application Prototype Flow

4.6 Summary

This chapter describes the design decision that was made before implementation began. The system architecture was kept straightforward. With a clear flow from user input through preprocessing and model prediction to output. Two separate preprocessing pipelines were designed to suit the different requirements of the LinearSVC and DistilBERT models. The LinearSVC was designed as a fast and efficient baseline that uses TF-IDF features, while DistilBERT was designed as an advanced model using transformer-based tokenisation and fine-tuning. The user interface was designed to be accessible to non-technical users while still giving detailed evaluation metrics. Chapter 5 now describes how all of these designs were implemented.

CHAPTER 5: IMPLEMENTATION

This chapter describes the full implementation of the fake news detection of the system as developed in Semester 2. Building on the prototype delivered in Semester 1, the implementation was expanded across three of the main components, which was the data preparation pipeline, the two machine learning models (LinearSVC and DistilBERT and the Streamlit web application. All code was written in Python. Each component is discussed in detail in the following sections.

5.1 Dataset Preparation

The dataset that was found from Kaggle which I obtained as a fake and real news dataset and was then used for training and testing both models. The dataset consists of two separate CSV files, Fake.csv and True.csv. A Python script called prepare_data.py was then written to combine these into a single dataset.

The script works as follows:

1. Load Fake.csv and True.csv using pandas
2. Add a label column, "FAKE," for Fake news and “REAL” for Real news
3. Extract only the headline (title column) from each file
4. Combine both files into one dataset
5. Shuffle the dataset randomly.
6. Save the combined dataset as data.csv

The final dataset contained approximately 44,000 labelled headlines, which were split evenly between Real and Fake news. The decision to use headlines rather than using full

articles was made to reduce complexity and reflect on real-world usage, where users often judge news based solely on headlines. This will also allow models to focus on the linguistic patterns that are commonly found in misleading or exaggerated headlines.

5.2 Linear Support Vector Machine Model Implementation

The LinearSVC model was implemented in model.py using scikit-learn, combining the TF-IDF vectoriser and classifier into a single pipeline.

The TF-IDF vectoriser was configured with the following settings:

- stop_words=”english” removes the most common words like “the” and "and"
- ngram_range= (1,2) learns single words and word pairs
- min_df=2 ignores very rare words
- max_df=0.9 ignores words that appear in almost every headline

The use of bigrams (ngram_range = (1,2)) allows the model to capture short phrases like “breaking news” or “shocking discovery”, which can be strong indicators of fake news. Additionally, filtering and overly common words helps to reduce noise and improves model generalisation.

The dataset was split into 67% training and 33% testing using train_test_split with a fixed random state of 42 to ensure that there are reproducible results. A 67/34 train-test split was selected to ensure that the model has sufficient data to learn the patterns while still maintaining a large and reliable test that is set for the evaluation. The use of a fixed random state ensures the reproducibility of results. The model was trained on the training

set and evaluated on the test set. The trained model was saved as fake_news_model. joblib using joblib so that it could be loaded by the web application without retraining.

5.3 DistilBERT Model Implementation

The DistilBERT model was fine-tuned using Google Colab with a T4 GPU. The base model used was distilbert-base-uncased from HuggingFace Transformers. DistilBERT was chosen over the full BERT model due to its reduced size and faster performance, while still retaining a high level of accuracy making it more suitable for deployment in a real-time application.

The fine-tuning process involved the following steps:

1. Load the Kaggle dataset and encode labels as 0 (Fake) and 1 (Real)

2. Tokenise the headlines using DistilBert TokenizerFast with maximum length of 128 tokens

3. Split the data into 67% training and 33% testing

4. Fine tune DistilBertForSequenceClassification model on train set

5. Evaluate the model on the test set

6. Save the fine-tuned model to a bert_model_folder

7. Download the model as a zip file to use it locally

A maximum sequence length of 128 tokens was selected, as headlines are generally short, which allows the model to process the input efficiently without any unnecessary padding or a computational overhead.

The model was trained on a GPU to speed up the fine-tuning process. After the training, the model files were downloaded from Google Colab in the bert_model folder in the project directory.

5.4 Streamlit Web Application

The web application was built using Streamlit and was implemented in app.py. The app allows users to interact with both models through a clean, dark-themed interface.

The main features of the application are:

- Text input - users can paste any news headline or article
- Model selector - users can choose between LinearSVC and DistilBERT
- Prediction result - displays REAL or FAKE with a clear colour-coded result
- Article statistics - shows word count and character count
- Model evaluation metrics - displays the accuracy, precision, recall, F1-score, and confusion matrix for both models in separate tabs.

The user interface was designed to be simple to ensure that non-technical users can easily interact with the system. A dark theme was used to improve readability and provide a modern, professional appearance.

The app loads both models using caching so they only need to be loaded once, which improves the performance. The LinearSVC model is loaded from fake_news_model.joblib and the DistilBERT model are loaded from the bert_model folder.

Without the use of caching, the models would need to be reloaded on every interaction, significantly increasing response time. The use of caching improves performance and ensures a smoother, more friendly user experience.

Overall, the web application transforms the machine learning models into a practical tool that can be used by in real -world scenarios, making the system accessible for everyday users.

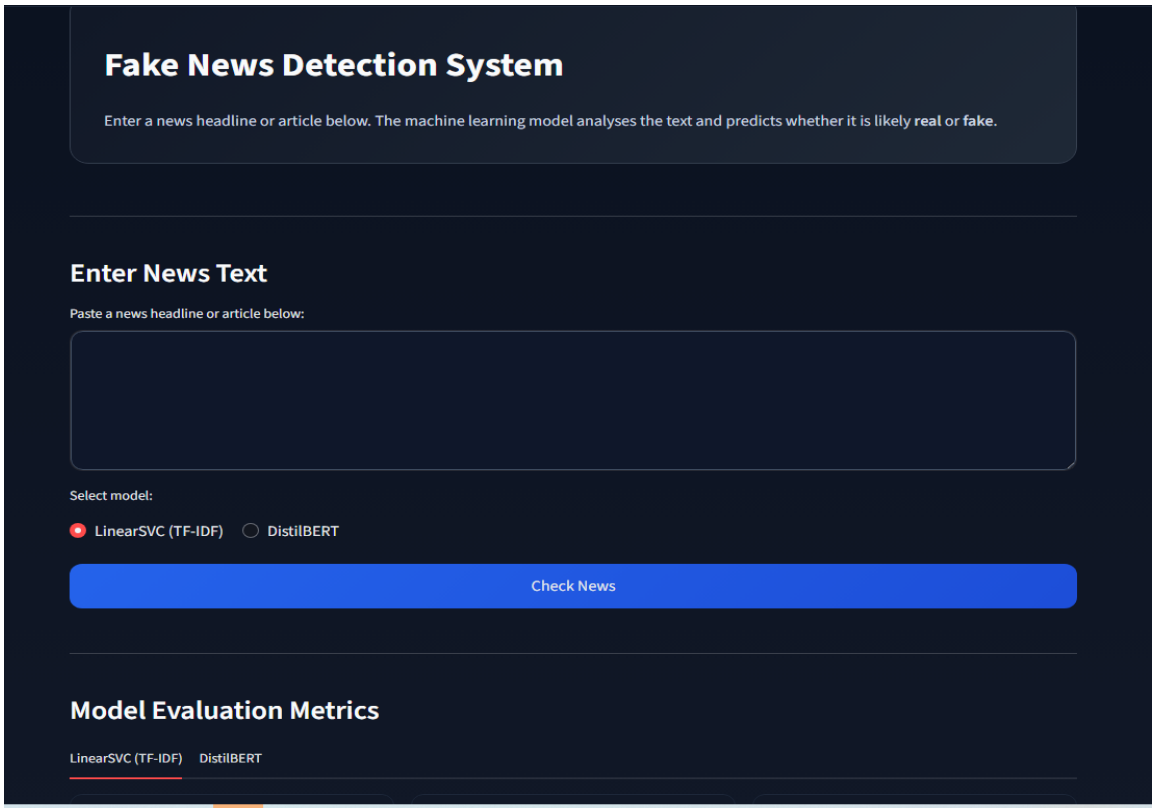

Figure 7: Streamlit Web Application.

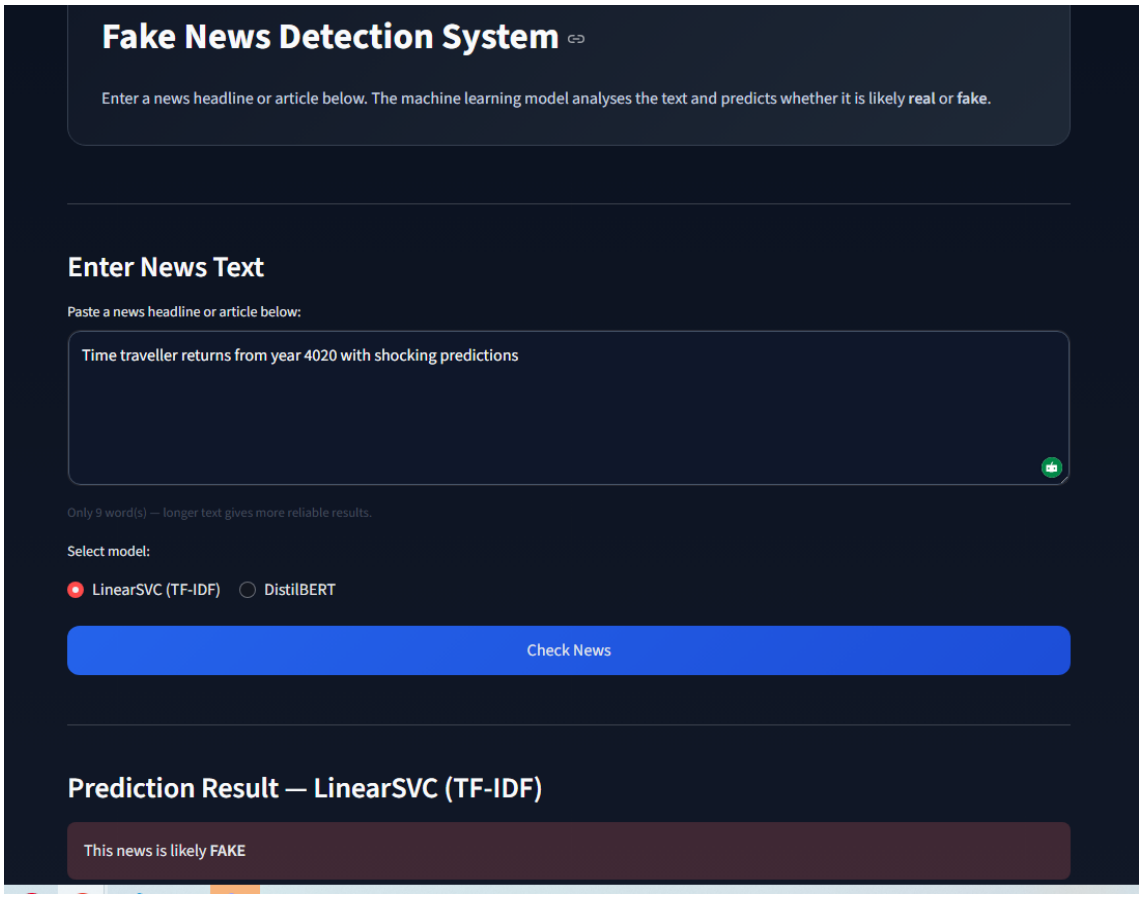

Figure 8: Streamlit Web Application

5.5 Summary

Overall, the implementation was successfully delivered, achieving all objectives that was set out in chapter 1. The dataset preparation pipeline combined and cleaned the Kaggle dataset, making it ready for training. The LinearSVC model was trained locally using scikit-learn and achieved an accuracy of 95.6%. The DistilBERT model was fine-tuned on Google Colab using T4 GPU and achieved 98.8% overall accuracy. Both of the models were also integrated into Streamlit web application, which allows users to get real-time predictions from either of the models.

CHAPTER 6: TESTING & EVALUATION

6.1 Overview

This chapter presents the testing and evaluation of both of my models that is developed in this project, the LinearSVC baseline model and the fine -tuned DistilBERT transformer model. Both of the models were evaluated on the same test that was held out by using accuracy, precision, recall, F1-score, and the confusion matrix analysis. The results are compared and discussed in relation to the project objectives set out in Chapter 1

6.2 Dataset Split

The full dataset contained 44,898 labelled headlines, divided into 30,081 for training (67%) and 14,817 for testing (33%).The split was done using scikit-learn's train_test_split with a fixed random state of 42, ensuring the same headlines are consistently assigned to the same set and making the results reproducible.

A 67/33 split was selected because the dataset is large enough that 14,000+ test headlines still provide reliable evaluation results. The dataset was also balanced, containing 23,481 Fake and 21,417 Real headlines, so no resampling techniques were necessary. Because of this, accuracy remained an appropriate metric to use alongside precision, recall and F1-score. This balance was preserved across both the training and testing sets after the split.

Both models were fitted using only the training data and then evaluated using the test data. The test set was never used during training, meaning the results in Sections 6.3 and 6.4 represent true performance on unseen data.

6.3 LinearSVC Model Results

The LinearSVC model achieved an overall accuracy of 95.6% on the held-out test set. The full results shown below

Table 3: LinearSVC Evaluation

Class	Precision	Recall	F1-Score
Fake	96%	95.5%	95.8%
Real	95.1%	95.7%	95.4%

Confusion Matrix

- True Fake (correctly predicted fake): 7,408
- False Real (fake predicted as real): 346
- False Fake (real predicted as fake): 307
- True Real (correctly predicted real): 6,756

The confusion matrix breaks down the model's prediction into four main categories. True Fake (7,408), which represents the number of fake headlines that the model correctly identified as fake. False Real (346), which represents the fake headlines that the model correctly classified as real. These are a sign of bad errors as they mean that fake content would go undetected. False Fake (307) represents the real headlines that were incorrectly flagged as fake. True Real (6,756) represents real headlines correctly identified as real. The model made a total of 653 errors out of 14,817 headlines, giving an overall error rate of 4.4%.

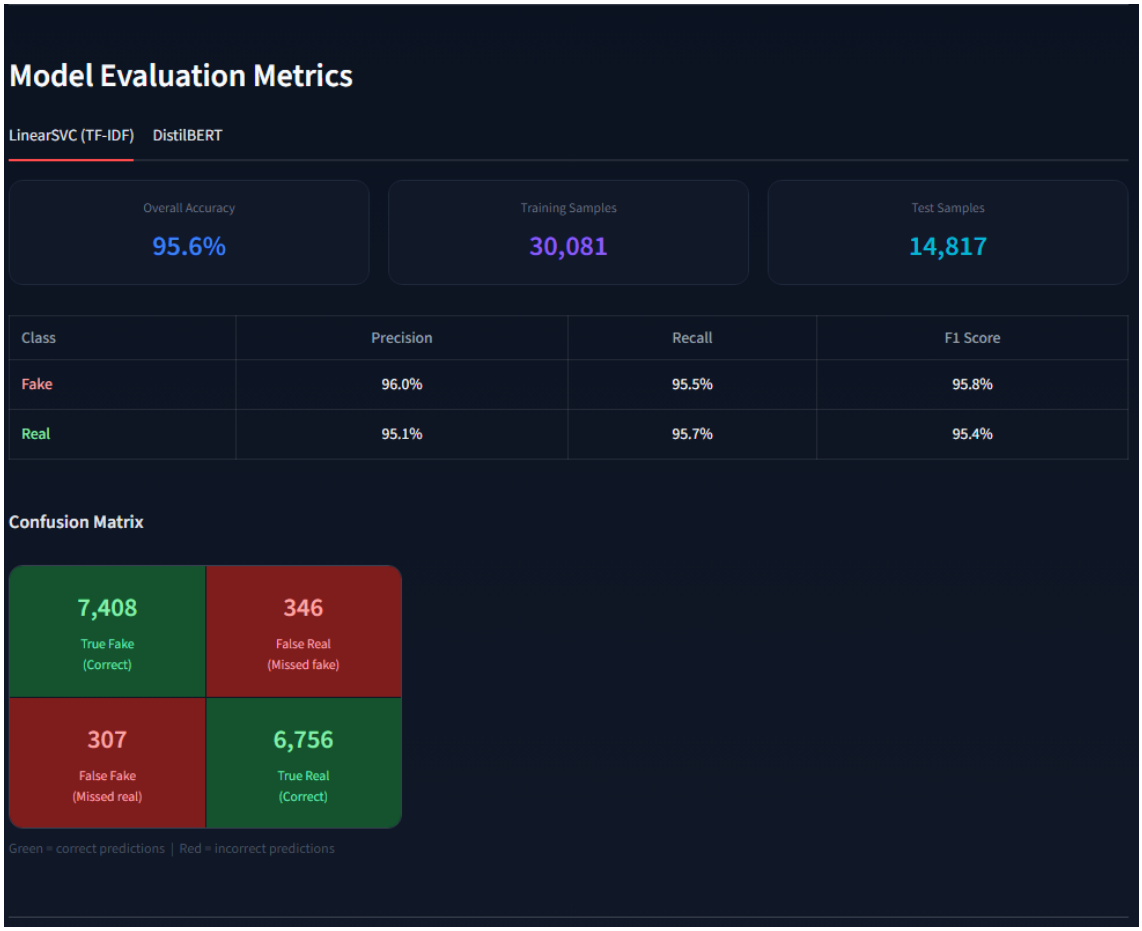

Figure 9: LinearSVC Model Evaluation Metrics in Streamlit

The results that are shown above that the LinearSVC model performs very well on the Kaggle dataset. The model is slightly better at identifying Fake news than Real news, with a higher precision score for the fake class. The main weakness is that 346 fake headlines were incorrectly classified as real, which highlights the limitation of TF-IDF in detecting realistic-sounding fake news.

6.4 DistilBERT model Results

The DistilBERT model achieved an overall accuracy of 98.8% on the same held-out test set. The full results are shown below:

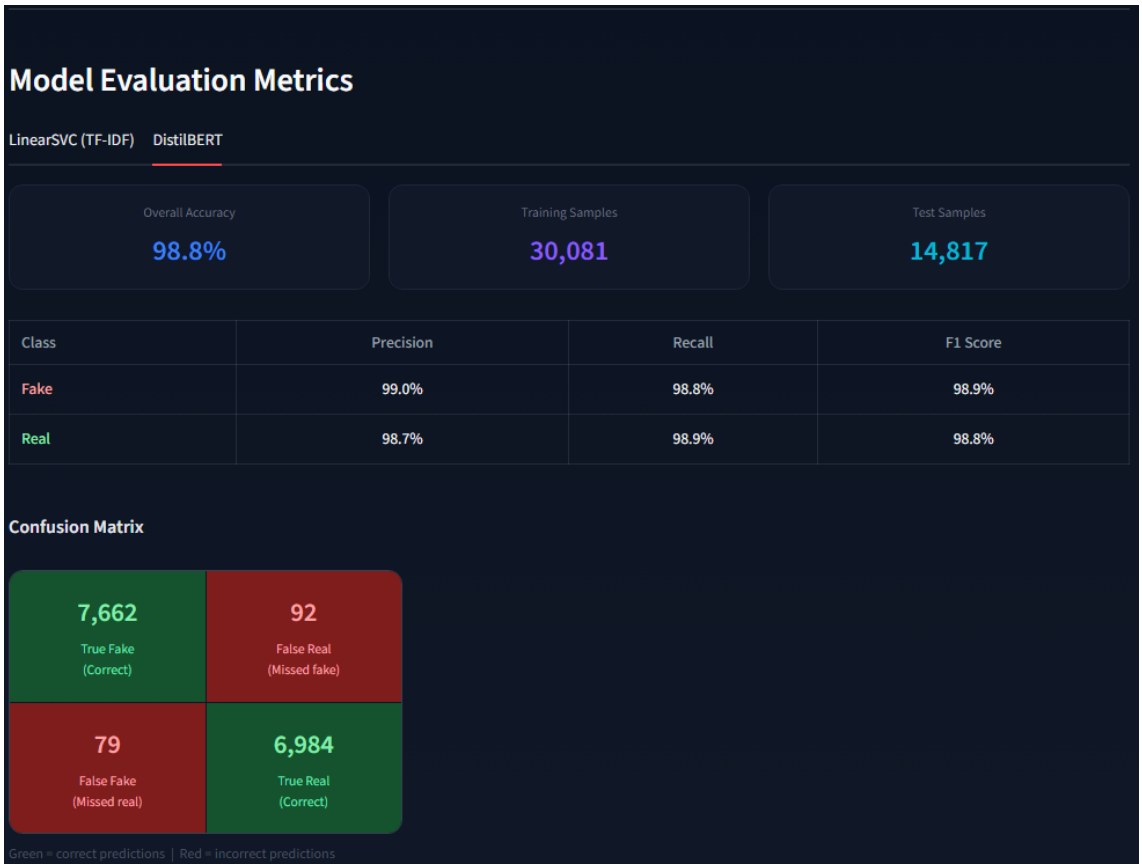

Figure 10: DistilBERT Model Evaluation Metrics in Streamlit

Table 4: DistilBERT Evaluation

Class	Precision	Recall	F1-Score
Fake	99.0%	98.8%	98.9%
Real	98.7%	98.9%	98.8%

Confusion Matrix:

True Fake (correctly predicted fake): 7,662

False Real (fake predicted as real): 92

False Fake (real predicted as fake): 79

True Real (correctly predicted real):6,984

The DistilBERT model significantly outperforms the LinearSVC baseline across all metrics. The number of missed fake lines dropped from 346 to just 92, which shows the clear advantage of using a transformer model that understands the context and meaning rather than just word frequency.

6.5 Model Comparison

Table 5: Model Comparison Table

Metric	LinearSVC	DistilBERT
Accuracy (Fake)	95.6%	98.8%
Precision (Fake)	96.0%	99.0%
Recall (Fake)	95.5%	98.8%
F1-Score (Fake)	95.8%	98.9%
Precision (Real)	95.1%	98.7%
Recall (Real)	95.7%	98.9%
F1-Score (Real)	95.4%	98.8%

DistilBERT outperforms LinearSVC across every metric; however, the LinearSVC model is still a strong model, as it is much faster to train and requires no GPU and achieves 95.6% accuracy, which is well above the Semester 1 baseline of 75%

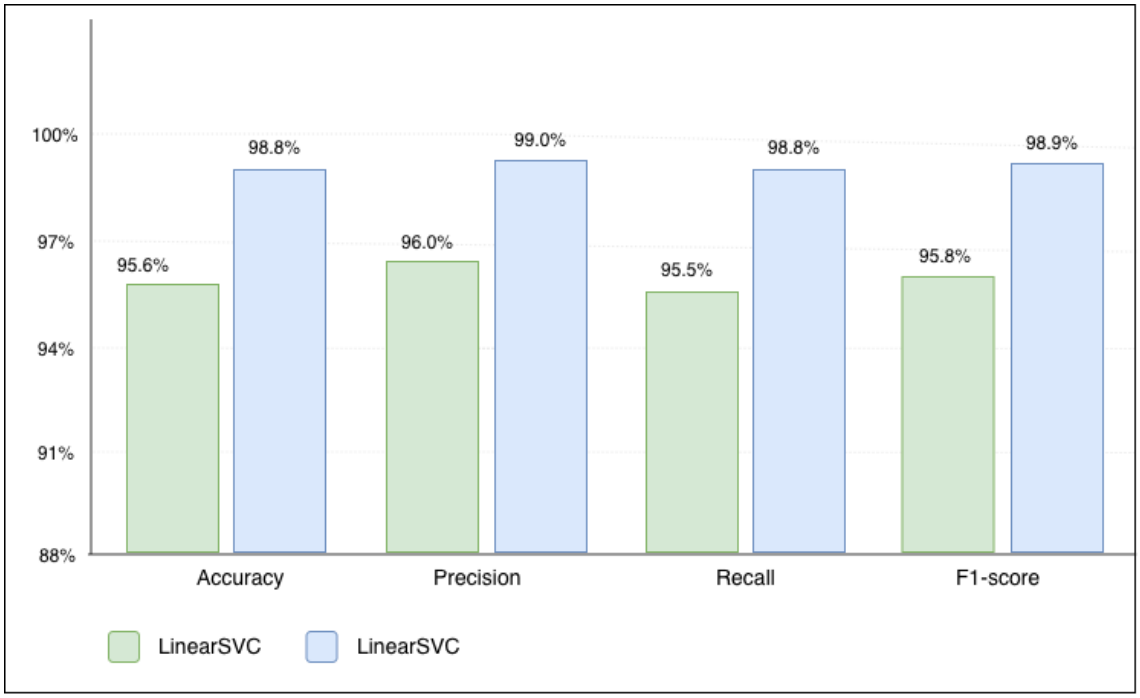

Figure 11: Model Comparison Chart

6.6 Manual Testing

Both models were also tested manually using the Streamlit web application. A selection of headlines was entered to test how each of the models handles different types of news.

Correctly predicted as Fake by both models:

“Man claims he can live without food for 10 years thanks to magic crystal”

“Time traveller returns from year 4020 with shocking predictions”

“Aliens spotted stealing cows in the Wicklow mountains”

“Correctly predicted as REAL by both models”

Correctly predicted as REAL by both models:

“Dublin Airport reports record number of passengers in 2025”

“Irish government unveils new healthcare plan to support elderly residents”

“New electric bus fleet launched in Dublin to reduce emissions”

Difficult cases:

“Researchers discover water on Mars in the latest NASA mission”- LinearSVC predicted FAKE, DistilBERT predicted REAL correctly. This shows how DistilBERT handles scientific headlines better than the TF-IDF baseline.

6.7 Evaluation Against Objectives

Table 6: Evaluation Against Project Objectives

Objective	Status
Collect and prepare a large, labelled dataset	44,000 headlines from Kaggle
Build and evaluate a LinearSVC model	95.6% accuracy
Fine-tune a DistilBERT model	98.8% accuracy
Evaluate using precision, recall, F1 and confusion matrix	Done for both of the models
Build a Streamlit web app	Fully functional
Critically evaluate strengths and weaknesses	Completed strengths and weaknesses, see sections 6.8 and 7.4

6.8 Summary

Overall, both of my models performed strongly on the Kaggle dataset. LinearSVC achieved 95.56% accuracy, and DistilBERT achieved 98.8% accuracy, which represents a significant improvement over Semester 1 baseline of 75%. DistilBERT outperformed

the LinearSVC across all of the metrics, which demonstrates the advantage of transformer-based models for fake news detection. All project objectives were successfully met.

CHAPTER 7: CONCLUSIONS & FUTURE WORK

7.1 Conclusions

This project set out to build a fake news detection system that uses text analysis and AI to classify news headlines if they are Real or Fake. Over the course of two semesters, the system evolved significantly from a basic console-based prototype to a fully functional web application with two machine learning models.

In Semester 1, a baseline prototype was built by using TF-IDF and a LinearSVC classifier, which achieved an overall 75% accuracy on a small custom dataset. While this was a solid starting point, the model struggled with realistic-sounding news, and it had limited generalisation due to the small dataset size.

In Semester 2, the system was significantly improved as the dataset was expanded a substantial amount from Kaggle Fake Real News dataset containing approximately 44,000 labelled headlines. The LinearSVC model was retrained on this larger dataset and achieved an accuracy of 95.6%. The DistilBERT transformer model was fine-tuned on the same dataset using Google Colab with a T4 GPU and achieved an accuracy of 98.8%. Both models were evaluated using accuracy, precision, recall, F1- score, and confusion matrix analysis.

7.2 Reflection

This project was a genuinely valuable learning experience, as it allowed to apply machine learning, natural language processing, and web development skills to a real-world problem that actually matters. Working through the full pipelines from collecting and cleaning a dataset to training and evaluating models to building a working web

application, gives a much better understanding of how these systems come together in practice rather than just theory.

The biggest challenge was fine-tuning the DistilBERT model, as it required understanding transformer architecture, tokenization, and GPU training on Google Colab, which consumed time, but the results made it worthwhile. The 3.2 percentage point accuracy gain over LinearSVC might not sound dramatic, but in terms of missed fake headlines on the test set, it meant the number of false negatives dropped from 346 to just 92, which in a real-world application would be a meaningful difference.

If I were starting this project again, I would invest a lot more time earlier in the semester on data exploration and on reading more recent papers on transformer -based approaches, as this would have helped me design the DistilBERT implementation more confidently from the start.

7.3 Limitations

Despite having very strong results, the system has several limitations that should be acknowledged:

- Both models were trained mainly on US politics, which means they may not generalise well to other topic such as health science or international news.

- The system only analyses text of the headlines. It cannot verify facts, check sources or consider the context in which a headline was shared. A headline that is technically accurate but deeply misleading would not be detected.

- DistilBERT is slower and more resource-intensive than the LinearSVC. In its current form it is not practical to run in low-powered devices a GPU.

- The suspicious word feature is rule-based and does not adapt to new language patterns. It could not become outdates quickly as the style of fake headlines evolves.

- The dataset is binary, Real or Fake, and cannot handle partially true stories or satire or context-dependent claims, which are some of the most difficult types of misinformation to detect

7.4 Future Work

If given more time, the following improvement could be made to extend the system:

- Using a larger and more diverse dataset that covers different topics such as health, science and international news would improve generalisation and make the system more useful beyond US political content

- Deploying the Streamlit application online using a service like Streamlit Cloud or AWS would make it accessible to the general public rather than requiring a local setup to run it.

- Adding explainability features like LIME or SHAP it would allow the system to highlight which words influence the prediction would make it more transparent and trustworthy.

- Extending the system to detect fake news in other languages would make it more useful globally, as it would increase usefulness, particularly given how widely misinformation spreads across different languages and regions.

- Analysing full articles rather than just headlines would likely improve accuracy, though it would increase complexity in terms of preprocessing pipeline and the computational requirements.

- Testing the system against deliberately manipulated headlines (adversarial testing) would help identify and fix weaknesses before they could be exploited in a real-world setting.

7.5 Summary

The project successfully delivered a fake news detection system that meets all of the six objectives that were set out at the start of the year. The system involved going from having a prototype that would give an overall accuracy of 75% in Semester 1 to a 98.8% accurate DistilBERT model with a fully functional Streamlit web application in Semester 2. The project demonstrates the clear advantage of transformer- based models over classical machine learning for fake news detection, while also acknowledging the limitations that remains. Fake News detection is a genuinely difficult, important problem and while this system is not a complete solution, it is meaningful step in the right direction.

APPENDIX A – PROJECT PLAN SEMESTER 1

Week	Dates	Work Completed
Week 1	Early September	Choose project topic and discussed ideas with supervisor
Week 2	Mid-September	Research fake news and how it spread
Week 3	Late September	Reviewed academic papers on fake news detection methods
Week 4	Early October	Looked at different datasets and selected one to use
Week 5	Mid-October	Started preparing the dataset and cleaned the text
Week 6	Late October	Learned how TF-IDF works and tested small examples
Week 7	Early November	Implemented TF-IDF and trained the first SVM model
Week 8	Mid November	Tested model accuracy and improved settings

Week 9	Late November	Built a console-based prototype to predict REAL/FAKE
Week 10	Early December	Wrote Testing Strategy and sections
Week 11	Late December	Completed interim report chapters and finals edits

APPENDIX B – PROJECT DIARY SEMESTER 2

Week	Dates	Work completed
Week 1	Late January	Met with supervisor to discuss Semester 1 feedback. Planned Semester 2 improvements
Week 2	Early February	Downloaded and explored Kaggle Fake and Real News dataset
Week 3	Mid-February	Built prepare_dataset.py to combine Fake.csv and True.csv into single dataset
Week 4	Late February	Retrained LinearSVC model on new dataset. Achieved 95.6% accuracy
Week 5	Early March	Researched DistilBERT and set up Google Colab environment with T4 GPU
Week 6	Mid-March	Fine-tuned DistilBERT on Kaggle dataset. Achieved 98.8% accuracy
Week 7	Late March	Downloaded fine-tuned DistilBERT model and integrated into project
Week 8	Early April	Built Streamlit web application with model selector and prediction display
Week 9	Mid-April	Added metrics display, suspicious words feature and styling to Streamlit app
Week 10	Late April	Wrote and completed final thesis chapters

Week 11	Late April	Prepared for final demonstration and submission

APPENDIX C – CODE LISTING

Semester 1 Code

Appendix C.1 – Model.py

```
#  model.py

# This file TEACHES the AI how to predict if a headline is REAL or FAKE


import pandas as pd          # handles the dataset

import joblib                # saves/loads the trained AI model

from sklearn.model_selection import train_test_split  # splits data into training + testing

from sklearn.pipeline import Pipeline            # allows multiple steps together

from sklearn.feature_extraction.text import TfidfVectorizer  # turns text into numbers

from sklearn.svm import LinearSVC                # our AI "brain"

from sklearn.metrics import accuracy_score        # measures how smart our AI is


MODEL_PATH = "fake_news_model.joblib" # where we will save the trained model


print (" Loading data.csv...")

df = pd.read_csv("data.csv") # read our dataset


# Splits the dataset into:

#   X = the headlines (text)

#   y = the labels (real/fake)
```

```
X = df["text"]
y = df["label"]

# Training where the AI learns examples
# Testing where we check if the AI learned correctly
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.33, # one-third of the data is for testing
    random_state=42   # makes the split the same every run
)

# Step 1: Convert text into TF-IDF numbers
# Step 2: Train Linear SVM model to classify real/fake
pipeline = Pipeline ([
    ("tfidf", TfidfVectorizer(stop_words="english")), # remove common useless words
    ("svc", LinearSVC())                # train the AI brain
])

print (" Training model...")
pipeline.fit(X_train, y_train)  # let the AI learn from data

# Test the AI to see how well it learned
y_pred = pipeline.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print (f" Prototype accuracy: {accuracy:.3f}") #

#  Save the trained AI
```

```
joblib.dump(pipeline, MODEL_PATH)
print (f"model saved as {MODEL_PATH}")
```

Appendix C.2 Detector.py

```
# loads the trained AI model
import joblib  # loads the trained AI model

MODEL_PATH = "fake_news_model.joblib"

print (f"Loading model from {MODEL_PATH} ...")
# Open the trained AI brain
model = joblib.load(MODEL_PATH)

# Welcome message for the user
print ("\n=== Fake News Detector Prototype ===")
print ("Type any news headline and press Enter.")
print ("Type 'exit' to quit.\n")

# Loop forever until user says exit
while True:
# asks the user for a headline
    text = input ("Enter headline > "). strip () # ask user for a headline
    # If user types exit it will stop the program
    if text.lower() == "exit":
        print("Goodbye!")
        break
```

```
    if not text:  # if user didn't type anything it will ask again to
        continue

    # Ask the AI model for a prediction
    prediction = model.predict([text]) [0]

    # Show the result in big bold text
    print ("Prediction:", prediction.upper(), "\n")
```

Semester 2 Code

Appendix C.3 Prepare_dataset.py

```
import pandas as pd

# Load the fake and real news CSV files
fake = pd.read_csv("Fake.csv")
true = pd.read_csv("True.csv")

# Add labels so the model knows which is fake and which is real
fake["label"] = "fake"
true["label"] = "real"

# Use only the headline (title) from each file
fake["text"] = fake["title"]
true["text"] = true["title"]

# Keep only the text and label columns
```

```
fake = fake [["text", "label"]]
true = true [["text", "label"]]

# Combine both of the datasets into one
df = pd.concat([fake, true])

# Shuffle the data randomly so it's not all fake then all real
df = df.sample(frac=1, random_state=42)

# Save the combined dataset as data.csv
df.to_csv("data.csv", index=False)
```

Appendix C.4 Model.py (LinearSVC Model Training)

```
#  Converts the text into numbers using TF-IDF
# Trains the LinearSVC model on those numbers
pipeline =          Pipeline ([
    ("tfidf", TfidfVectorizer(
        stop_words="english", # ignore common words like: ‘the’, ‘and’
        ngram_range= (1,2), # learn single words and word pairs
        min_df=2, # ignores the very rare words
        max_df=0.9          # ignore words that appear in almost every headline
    )),
    ("svc", LinearSVC()) # the AI classifier
])

# Train the model on the training data
pipeline.fit(X_train, y_train)
```

```
# Save the trained model so we can use it in the web app

joblib.dump(pipeline, MODEL_PATH)
```

Appendix C.5 – app.py (DistilBERT prediction in Streamlit)

```
def predict_bert(text): # Load the tokenizer and model tokenizer, model = load_bert()

# Convert the headline into tokens so the model can understand
inputs = tokenizer(text, return_tensors="pt",
truncation=True, padding=True, max_length=128)

# Run the model without updating its weights
with torch.no_grad():
    outputs = model(**inputs)

# Get the class with the highest score (fake or real)
pred_id = torch.argmax(outputs.logits, dim=1).item()

# Return the label (fake or real)
return model.config.id2label[pred_id]
```

Appendix C.6 App.py (LinearSVC prediction)

```
def predict_svm(text):

   # Load the saved LinearSVC model

   model = load_svm()


   # Use the model to predict if the headline is fake or real

   return model.predict([text]) [0]
```

REFERENCES

Ahmed, H., Traore, I. & Saad, S. (2018) ‘Detection of online fake news using n-gram analysis and machine-learning techniques’, Proceedings of the International Conference on Cyber Security and Protection of Digital Services, pp. 1–8

Ajao, O., Bhowmik, D. & Zargari, S. (2019) ‘Fake news identification on Twitter with hybrid CNN and RNN models’, Proceedings of the 9th International Conference on Social Media and Society, pp. 226–230.

Alshuwaier, F. & Alsulaiman, A. (2025) ‘Explainable AI methods for trustworthy misinformation detection’, Journal of Information Security, 14(2), pp. 55–70.

Alsulaiman, A. (2025) ‘Multilingual misinformation detection: Challenges and future directions’, International Journal of Computational Linguistics, 12(1), pp. 33–49.

Cui, L. & Lee, D. (2020) ‘CoAID: COVID-19 healthcare misinformation dataset’, arXiv preprint, arXiv:2006.00885.

Gong, Z., Liu, J., Wen, Q. & He, W. (2024) ‘Improving fake news detection with hybrid credibility signals and explainable AI’, Information Processing & Management, 61(3), pp. 1– 15.

Hu, X. (2025) ‘Lightweight LLM fine-tuning for misinformation detection’, AI Review, 38(1), pp. 112–129.

Hu, X., Wang, Y., Li, T. & Chen, J. (2024) ‘Evaluating large language models for fake news detection’, Proceedings of the IEEE International Conference on Big Data, pp. 512–520.

Kaliyar, R., Goswami, A. & Narang, P. (2021) ‘FakeBERT: Fake news detection in social media with transformer models’, Expert Systems with Applications, 174, p. 114.

Kumari, P. & Singh, R. (2025) ‘A comparative evaluation of metrics for fake news classifiers’, Machine Learning Analytics Journal, 9(1), pp. 45–60.

Oshikawa, R., Qian, J. & Wang, W.Y. (2020) ‘A survey on natural language processing for fake news detection’, Proceedings of the 12th Language Resources and Evaluation Conference, pp. 6086–6093.

Panwar, S., Gupta, A. & Yadav, P. (2024) ‘Limitations of accuracy in misinformation detection: Why multiple metrics matter’, Journal of Data Science Research, 15(2), pp. 89–104.

Patwa, P. et al. (2021) ‘Fighting an infodemic: COVID-19 fake news dataset’, Proceedings of the International Workshop on Combating Online Hostile Posts, pp. 1–7.

Raza, S., Ahmed, Z., Noor, S. & Malik, A. (2024) ‘A survey on transformer and LLM-based fake news detection models’, ACM Computing Surveys, 56(7), pp. 1–33.

Shu, K., Sliva, A., Wang, S., Tang, J. & Liu, H. (2017) ‘Fake news detection on social media: A data mining perspective’, ACM SIGKDD Explorations Newsletter, 19(1), pp. 22–36.

Shu, K., Wang, S. & Liu, H. (2020) ‘Disinformation, misinformation and fake news in social media’, Synthesis Lectures on Information Concepts, Retrieval, and Services, 11(3), pp. 1–95.

Shu, K. et al. (2017) ‘FakeNewsNet: A data repository with news content and social context’, arXiv preprint, arXiv:1809.01286.

Wang, W.Y. (2017) ‘“Liar, Liar Pants on Fire”: A new benchmark dataset for fake news detection’, Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, pp. 422–426.

Zhou, X. & Zafarani, R. (2020) ‘A survey of fake news: Fundamental theories, detection methods, and opportunities’, ACM Computing Surveys, 53(5), pp. 1–40.